

J Unit Testing Exercises

Exercise 1: Setting Up JUnit

Scenario:

You need to set up JUnit in your Java project to start writing unit tests.

Steps:

1. Create a new Java project in your IDE (e.g., IntelliJ IDEA, Eclipse).
2. Add JUnit dependency to your project. If you are using Maven, add the following to your pom.xml:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

3. Create a new test class in your project.

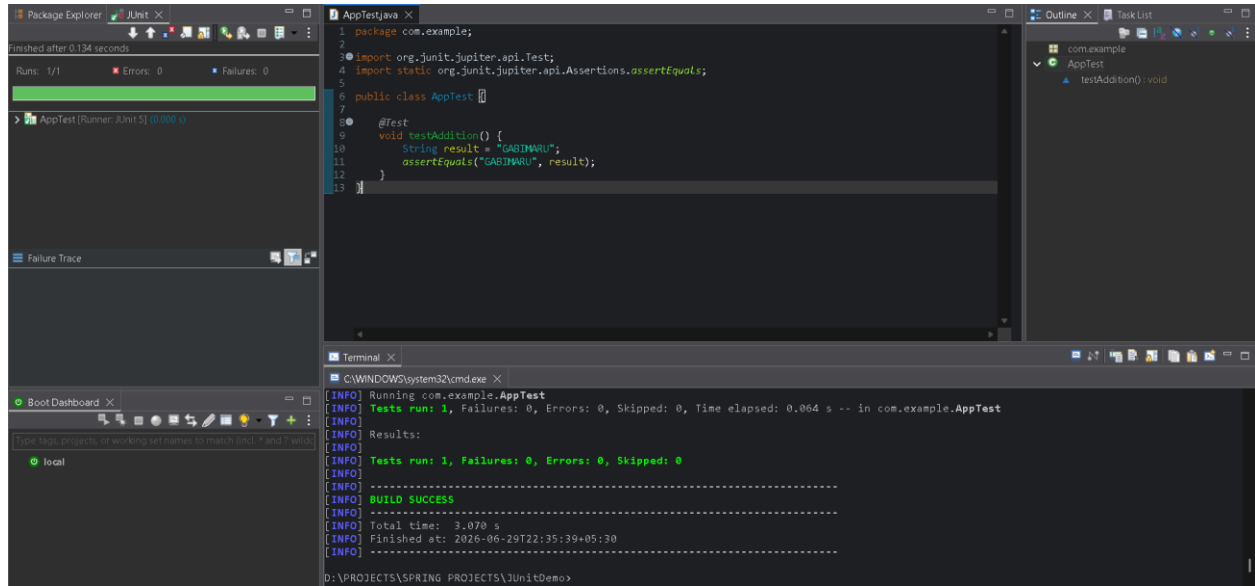
Solution:

1) Created the Project :

Picked "maven-archetype-quickstart" : Version 1.5
Name of the Project : JUnitDemo
GroupId/PackageId : com.example

2) Since there are Default dependencies of JUNIT in the MAVEN pom.xml FILE I DIDN'T USE THE 4.13.2 version. The default version is way better than the 4.13.2 . So to avoid the errors I used the default dependencies.

3)



Exercise 3: Assertions in JUnit

Scenario:

You need to use different assertions in JUnit to validate your test results.

Write tests using various JUnit assertions.

Solution:

```
package com.example;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class AssertionsTest {

    @Test
    void passingTest() {

        int a = 10;
        int b = 5;
        String name = "Abhiram";
        String empty = null;
        assertEquals(15, a + b);
        assertTrue(a > b);
        assertFalse(a < b);
        assertNull(empty);
        assertNotNull(name);
        assertNotEquals(20, a + b);
        assertSame(name, name);
        assertNotSame(name, new String("Abhiram"));
        assertEquals(new int[]{1, 2, 3}, new int[]{1, 2, 3});
        assertThrows(ArithmeticException.class, () -> {
            int x = 10 / 0;
        });
    }
}
```

```

@Test
void failingTest() {

    assertEquals(10, 2 + 3);
    assertTrue(5 < 3);
    assertFalse(5 > 3);
    assertNull("Hello");
    assertNotNull(null);
    assertNotEquals(5, 2 + 3);
    assertSame(new String("Java"), new String("Java"));
    assertEquals(new int[]{1, 2}, new int[]{1, 2, 3});
}

```

Output:

The screenshot displays an IDE interface with the following components:

- Package Explorer:** Shows the project structure with a 'JUnit' folder containing 'AssertionsTest.java'.
- JUnit Runner:** Displays test results for 'AssertionsTest' (Runner: JUnit 5) with a duration of 0.034 s. It shows 2/2 runs, 0 errors, and 1 failure. The failure is for the 'failingTest()' method, which took 0.029 s.
- Failure Trace:** Shows the stack trace for the failed test, indicating an 'AssertionFailedError' with the message 'expected: <10> but was: <5>'. The trace includes the following lines:


```

at org.junit.jupiter.api.AssertionFailedError.build(AssertionFailedError.java:33)
at com.example.AssertionsTest.failingTest(AssertionsTest.java:33)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)

```
- Code Editor:** Shows the source code for 'AssertionsTest.java'. The code includes imports for JUnit 5 assertions and a test method 'passingTest()' that passes all assertions. The 'failingTest()' method is commented out, showing the same assertions as the code block above.


```

package com.example;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class AssertionsTest {

    @Test
    void passingTest() {
        int a = 10;
        int b = 5;
        String name = "Abhiram";
        String empty = null;

        assertEquals(15, a + b);
        assertTrue(a > b);
        assertFalse(a < b);
        assertNull(empty);
        assertNotNull(name);
        assertNotEquals(00, a + b);
        assertEquals(name, name);
        assertNotSame(name, new String("Abhiram"));
        assertEquals(new int[]{1, 2, 3}, new int[]{1, 2, 3});
        assertThrows(ArithmeticException.class, () -> {
            int x = 10 / 0;
        });
    }
}

```
- Terminal:** Shows the output of the JUnit test run:


```

[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 2.582 s
[INFO] Finished at: 2026-06-29T22:48:26+05:30

```

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Scenario:

You need to organize your tests using the Arrange-Act-Assert (AAA) pattern and use setup and teardown methods.

Steps:

1. Write tests using the AAA pattern.
2. Use `@Before` and `@After` annotations for setup and teardown methods.

Solution:

CalculatorTest.java

```
package com.example;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class CalculatorTest {

    private Calculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new Calculator();
        System.out.println("Setup completed");
    }

    @Test
    void testAddition() {

        // Arrange
        int a = 10;
        int b = 5;

        // Act
        int result = calculator.add(a, b);

        // Assert
        assertEquals(15, result);
    }
}
```

```

@AfterEach
    void tearDown() {
        calculator = null;
        System.out.println("Cleanup completed");
    }
}

```

Calculator.java

```

package com.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}

```

Output:

